Buffer Overflow

Secure Programming

Michael Lehmann

Exam

- Dauer
  - 1.5h
- Inhalt
  - Secure Requirements Engineering
  - Secure Programming
- Form
  - Analog, i.e. auf Papier
  - Multiple Choice, offene Textaufgaben, Übungen analog Unterricht
- Rahmenbedingungen
  - Open book: Skript und Notizen digital oder analog – nichts anderes
  - Kein Internet oder Kommunikation

Security was not a concern when internet was developped

Development of the internet

- From ARPANET to the World Wide Web

Base principle

- Worldwide exchange of information between computers

Challenges

- Anonymity, data leaks, cyber criminals

There are different types of malware

- Ransomware
- Spyware
- Adware
- Trojan
- Worm
- Virus
- …and more

Morris developed the first worm

- First internet worm to count number of machines
- How do you go about this?

→ create a program to discover machines and copy itself

- Several techniques used, among them:
  - Dictionary of passwords
  - Buffer overflow

A buffer eventually overflows if input length is not checked

c

```c
#include <stdio.h>
#include <string.h>

int main() {
    char buffer[4];

    printf("Enter your input: ");
    gets(buffer);
    printf("You entered: %s\n", buffer);

    return 0;
}
```

yaml

```yaml
Enter your input: abc
You entered: abc
```

yaml

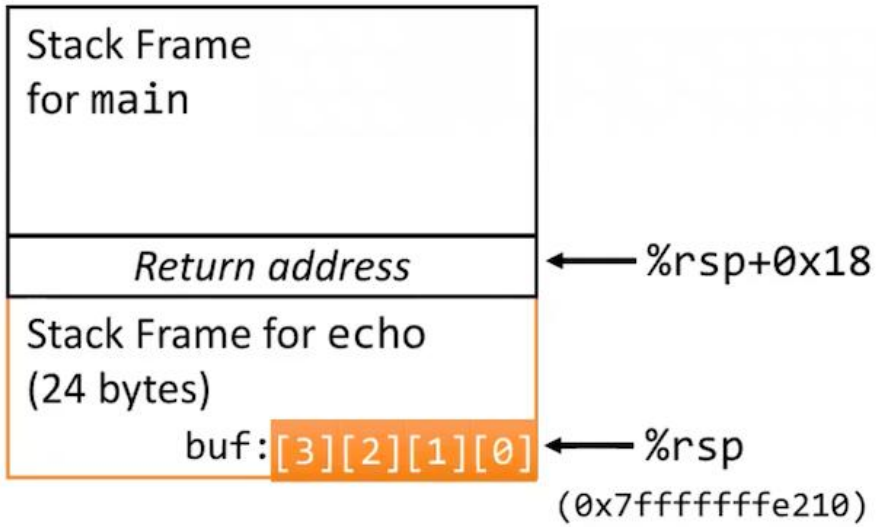
```yaml
Enter your input: abcde
You entered: abcde
```

less

```less
Enter your input: abcdefghijklmnopqrstuvwxyz
[Segmentation fault (core dumped)]
```

Let’s check out what happens

Before call to gets

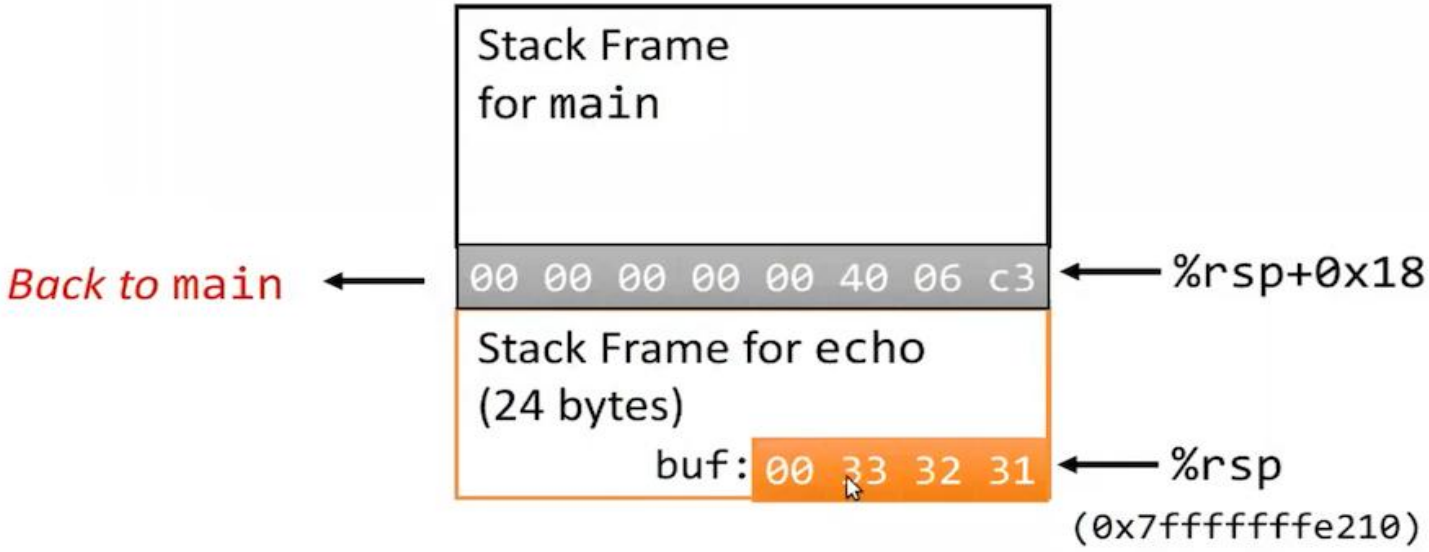



```
/* Echo Line */
void echo()
{
    char buf[4];
    gets(buf);
    puts(buf);
}
```

```
echo:
    subq    $24, %rsp
    movq    %rsp, %rdi
    call    gets
    movq    %rsp, %rdi
    call    puts
    addq    $24, %rsp
    ret
```

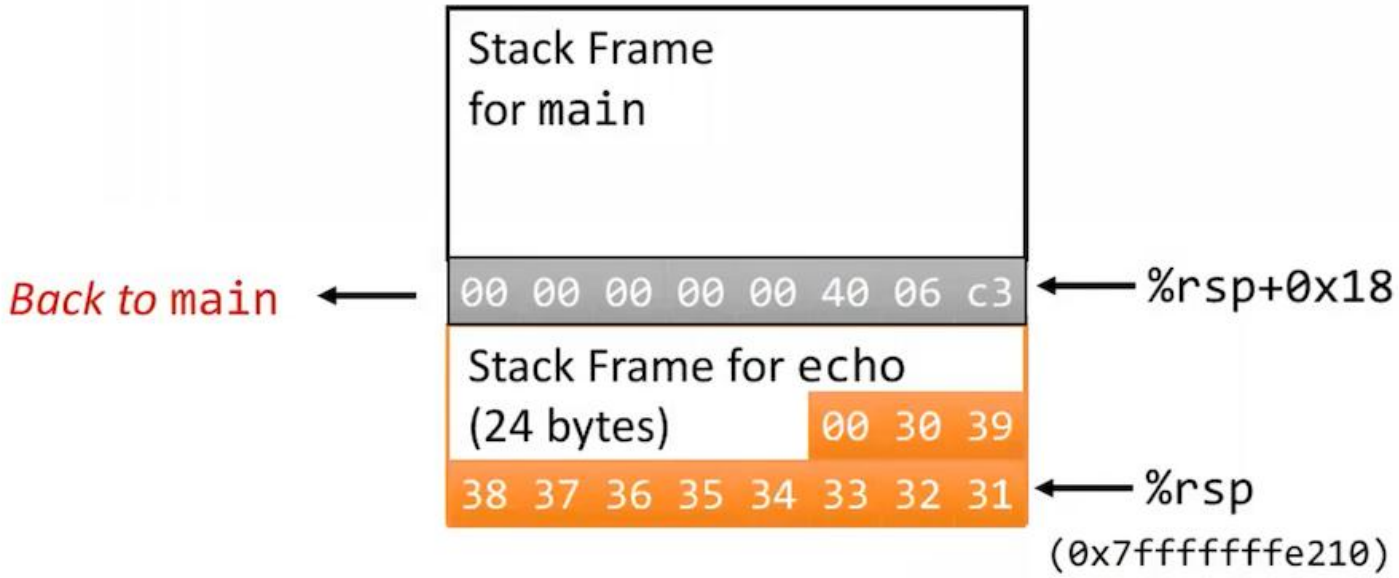
Stack holds input…

Input: 123



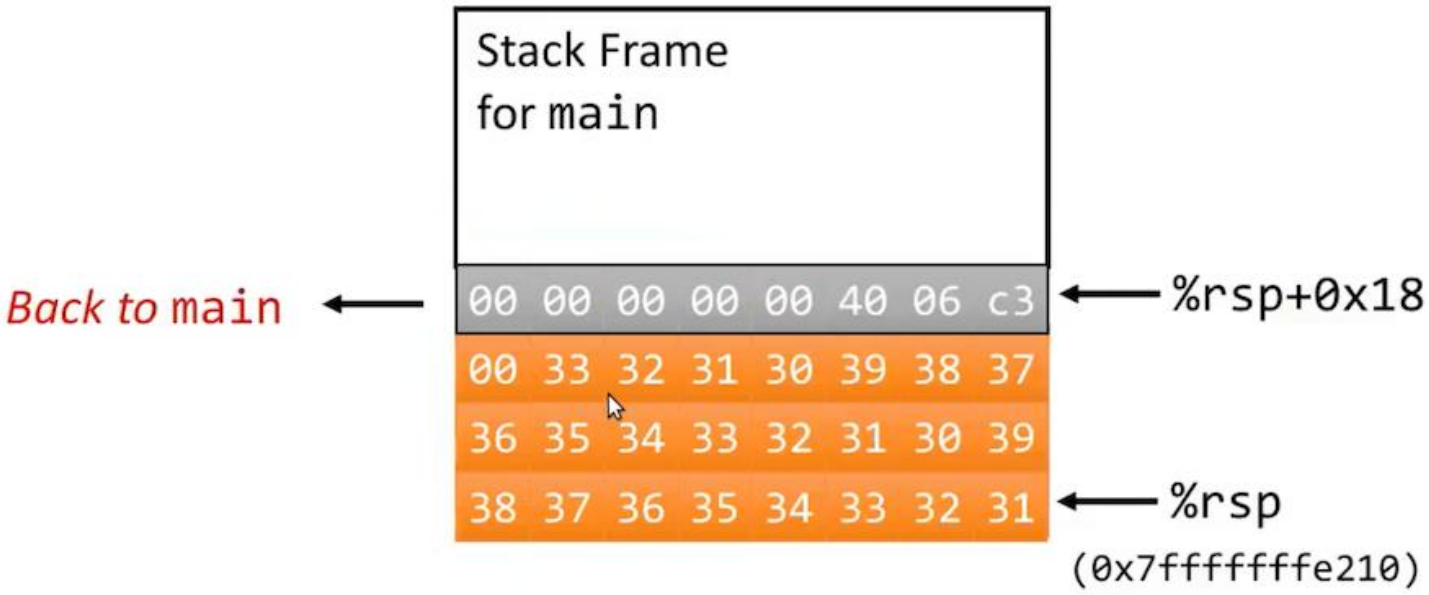
…still does…

Input: `123456790`



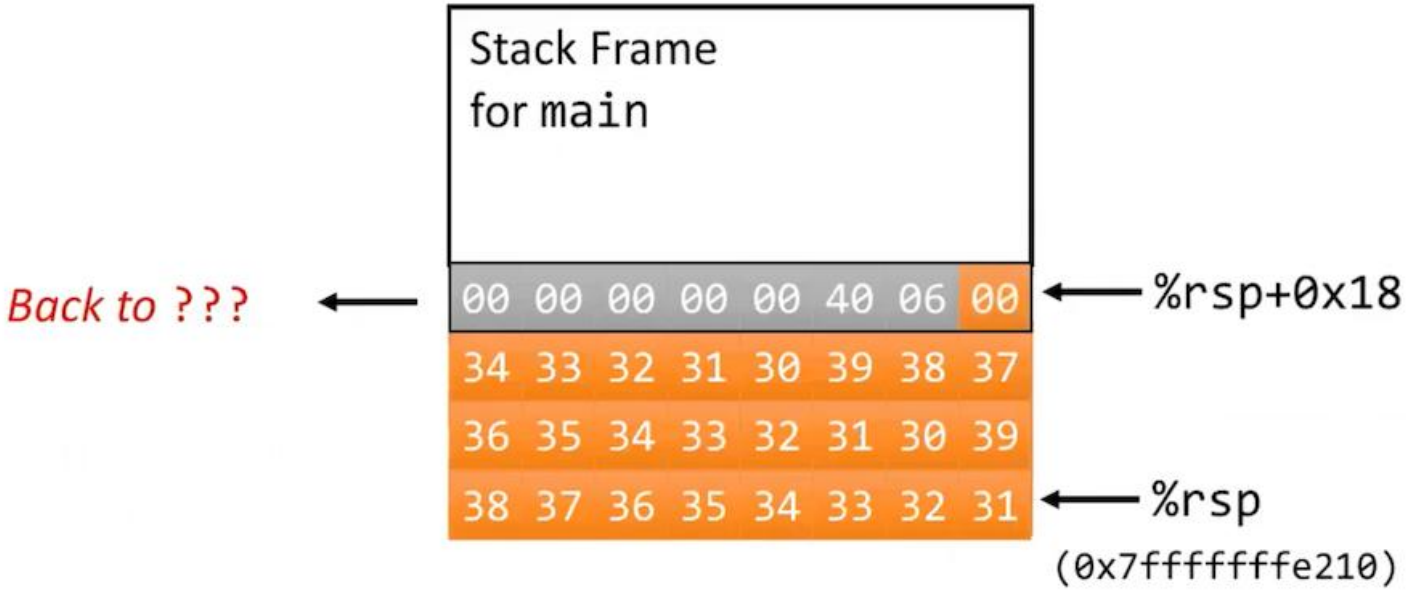
…barely…

Input: 1234567901234567901 23



…until it overflows

Input: 1234567901234567901234

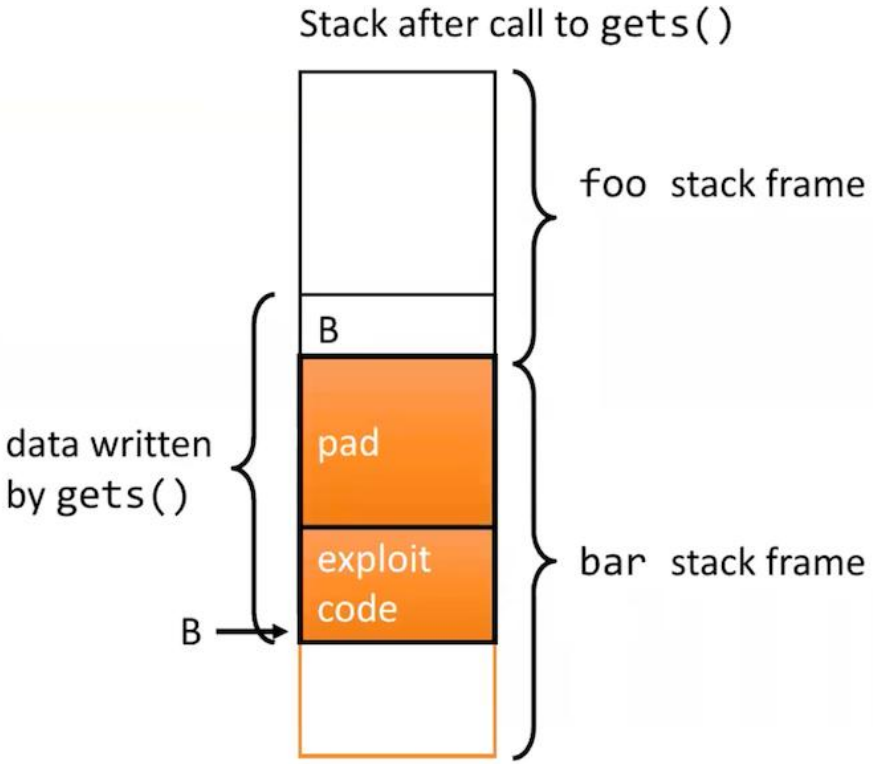


This can be exploited to run code

```
void foo(){
  bar();
  ...
}
```

return address A

```
int bar() {
  char buf[64];
  gets(buf);
  ...
  return ...;
}
```



Built-in protections exist…

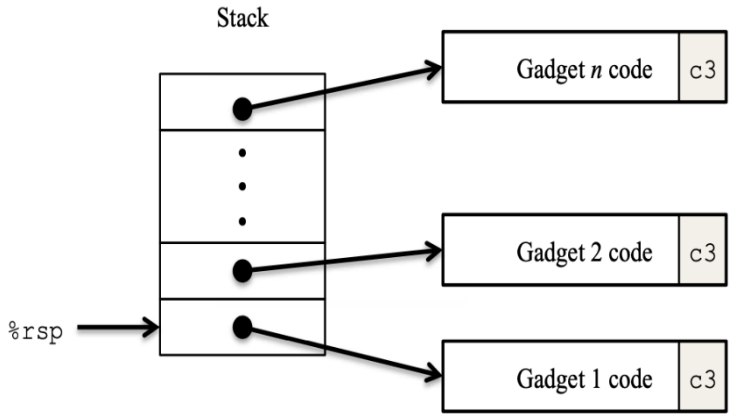
- Compiler-inserted checks on functions
- Address Space Layout Randomisation (ASLR)
- Stack canaries
- Non-executable code segments
  - Older x86: region of memory either «read-only» or «writeable»
    - Anything readable can be executed
  - Explicit «execute» permission added

…but also work-arounds to some

Return-oriented Programming (ROP)

- Leverages instruction sequences already present in executable memory
- The attacker manipulates return addresses and chains these «gadgets»
- Typical data execution prevention implementations cannot defend against this

→ why not?



What you can do

- Use languages that handle memory management
- If not..
  - ALWAYS check input length
  - Use safe functions (no gets, strcpy etc.)
  - Use modern compilers and their options
    - E.g. stack canaries, DEP/NX support, ASLR and more

Excercises

- https://overthewire.org/wargames/natas/

- Optional: https://overthewire.org/wargames/narnia/

Code Beispiel

- Was für eine Schwachstelle ist im folgenden Code Beispiel vorhanden und wie könnte diese verhindert werden (kein Code, nur Beschreibung der Mitigation)? (4)

php Copy code

```php
<?php
$page = $_GET['page'];
include($page . '.php');
?>
```

Beispiel CSRF

1) Damit CSRF potenziell möglich ist, muss/müssen Voraussetzung(en) gegeben sein. Dies(e) ist/sind (2):
   a) Das Session Cookie hat ein Attribut SameSite=Strict gesetzt
   b) Authentisierungs bzw. Session-Informationen werden mit jedem Request mitgeschickt
   c) Der User hat CSRF-Tokens blockiert
   d) Javascript ist nicht deaktiviert
   e) Keine

Beispiel DOM XSS

1) Welche der folgende(n) Aussage(n) über DOM-based XSS trifft/treffen zu (2)
   a) Die Payload wird nicht zum Server geschickt
   b) Ist immer Client-side
   c) Betrifft ein Plugin im Browser und daher alle Webseiten
   d) Ist immer eine reflected XSS
   e) Keine der Aussagen trifft zu